

GIÁO TRÌNH CAO CẤP MASTER EDITION

STACK & QUEUE IN C

Kiến trúc bộ nhớ, Thiết kế giải thuật và Tối ưu hóa hiệu năng

❏ GHI NHỚ: Tài liệu học tập chuyên sâu

Giáo trình được thiết kế đặc biệt dành cho sinh viên CNTT, kỹ sư phần mềm hệ thống và những ai mong muốn làm chủ cấu trúc dữ liệu tuyến tính từ bản chất phần cứng lên đến logic phần mềm. Có tham chiếu và kế thừa từ tài liệu gốc **"3. slide Stack Queue.pdf"**.

Giáo sư biên soạn: Giáo sư Cấu trúc dữ liệu & Software Engineer (30+ năm kinh nghiệm C)

Năm xuất bản: 2026

Mục lục

1	PHẦN I: STACK (NGĂN XẾP)	3
1.1	1. Khái niệm & Đặc tả toán học	3
1.2	2. Nguyên lý hoạt động LIFO (Last-In, First-Out)	3
1.3	3. Biểu diễn Stack bằng mảng (Array Representation)	3
1.4	4. Biểu diễn Stack bằng danh sách liên kết đơn (Singly Linked List Representation)	3
1.5	5. Thao tác Khởi tạo Stack (Stack Initialization)	3
1.5.1	1. FLOWCHART KHỞI TẠO	3
1.5.2	2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC	4
1.5.3	3. SƠ ĐỒ ASCII SAU THAO TÁC	4
1.5.4	4. PHÂN TÍCH THAY ĐỔI BIẾN	4
1.5.5	5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)	4
1.5.6	6. CODE C CHUẨN KHỞI TẠO	4
1.5.7	7. CHECKLIST 30 GIÂY	5
1.5.8	8. SAI LẦM THƯỜNG GẶP	5
1.5.9	9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O	5
1.5.10	10. MẸO GHI NHỚ	5
1.5.11	11. FLASHCARD CHUYÊN SÂU	5
1.5.12	12. TÓM TẮT THAO TÁC KHỞI TẠO	5
1.6	6. Thao tác Push (Thêm phần tử vào đỉnh Stack)	6
1.6.1	1. FLOWCHART PUSH	6
1.6.2	2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC	6
1.6.3	3. SƠ ĐỒ ASCII SAU THAO TÁC	6
1.6.4	4. PHÂN TÍCH THAY ĐỔI BIẾN	6
1.6.5	5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)	7
1.6.6	6. CODE C CHUẨN PUSH	7
1.6.7	7. CHECKLIST 30 GIÂY	7
1.6.8	8. SAI LẦM THƯỜNG GẶP	8
1.6.9	9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O	8
1.6.10	10. MẸO GHI NHỚ	8
1.6.11	11. FLASHCARD CHUYÊN SÂU	8
1.6.12	12. TÓM TẮT THAO TÁC PUSH	8
1.7	7. Thao tác Pop (Lấy và xóa phần tử khỏi đỉnh Stack)	9
1.7.1	1. FLOWCHART POP	9
1.7.2	2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC	9
1.7.3	3. SƠ ĐỒ ASCII SAU THAO TÁC	9
1.7.4	4. PHÂN TÍCH THAY ĐỔI BIẾN	9
1.7.5	5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)	10
1.7.6	6. CODE C CHUẨN POP	10
1.7.7	7. CHECKLIST 30 GIÂY	10
1.7.8	8. SAI LẦM THƯỜNG GẶP	10
1.7.9	9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O	11
1.7.10	10. MẸO GHI NHỚ	11
1.7.11	11. FLASHCARD CHUYÊN SÂU	11
1.7.12	12. TÓM TẮT THAO TÁC POP	11
1.8	8. Thao tác hỗ trợ Stack: Peek, isEmpty, isFull	12
1.9	9. Ứng dụng thực tế của Stack	12
1.10	10. Các lỗi lập trình viên thường gặp với Stack	13

2	PHẦN II: QUEUE (HÀNG ĐỢI)	14
2.1	1. Khái niệm & Đặc tả toán học	14
2.2	2. Nguyên lý FIFO (First-In, First-Out)	14
2.3	3. Biểu diễn Queue bằng Mảng (Array Representation)	14
2.4	4. Biểu diễn Queue bằng danh sách liên kết đơn (Singly Linked List Representation)	14
2.5	5. Thao tác Khởi tạo Queue	14
2.5.1	1. FLOWCHART KHỞI TẠO QUEUE	14
2.5.2	2. SƠ ĐỒ ASCII TRƯỚC VÀ SAU THAO TÁC	15
2.5.3	3. CODE C CHUẨN KHỞI TẠO QUEUE	15
2.5.4	4. CHECKLIST 30 GIÂY	15
2.6	6. Thao tác EnQueue (Thêm phần tử vào cuối hàng đợi)	16
2.6.1	1. FLOWCHART ENQUEUE	16
2.6.2	2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC ENQUEUE	16
2.6.3	3. SƠ ĐỒ ASCII SAU THAO TÁC ENQUEUE	16
2.6.4	4. PHÂN TÍCH THAY ĐỔI BIẾN	17
2.6.5	5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)	17
2.6.6	6. CODE C CHUẨN ENQUEUE	17
2.6.7	7. CHECKLIST 30 GIÂY	17
2.6.8	8. SAI LẦM THƯỜNG GẶP	18
2.6.9	9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O	18
2.6.10	10. MẸO GHI NHỚ	18
2.6.11	11. FLASHCARD CHUYỂN SÂU	18
2.6.12	12. TÓM TẮT THAO TÁC ENQUEUE	18
2.7	7. Thao tác DeQueue (Lấy phần tử ra khỏi đầu hàng đợi)	19
2.7.1	1. FLOWCHART DEQUEUE	19
2.7.2	2. SƠ ĐỒ ASCII TRƯỚC VÀ SAU DEQUEUE	19
2.7.3	3. CODE C CHUẨN DEQUEUE	19
2.7.4	4. CHECKLIST 30 GIÂY	20
2.7.5	5. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O DEQUEUE	20
2.8	8. Circular Queue (Hàng đợi vòng tròn)	21
2.9	9. Priority Queue (Hàng đợi ưu tiên)	21
2.10	10. Ứng dụng của Queue trong thực tế	22
3	PHẦN III: CHƯƠNG TỔNG HỢP & SO SÁNH TOÀN DIỆN	23
3.1	1. So sánh chi tiết triết lý Stack và Queue	23
3.2	2. Ưu và nhược điểm khi lựa chọn Array vs Linked List để cài đặt	23
3.3	3. Bảng tổng hợp độ phức tạp thời gian (Time Complexity) và không gian (Space)	23
3.4	4. Khi nào nên áp dụng loại nào?	24

1 PHẦN I: STACK (NGĂN XẾP)

1.1 1. Khái niệm & Đặc tả toán học

Stack (Ngăn xếp) là một cấu trúc dữ liệu tuyến tính đặc biệt hoạt động theo nguyên lý nghiêm ngặt: phần tử nào được thêm vào sau cùng sẽ là phần tử đầu tiên được lấy ra khỏi danh sách.

Trong tài liệu tham chiếu "**3. slide Stack Queue.pdf**", trang 1 mô tả trực quan hình ảnh một chồng sách lớn: cuốn sách đặt sau cùng ở trên đỉnh chồng sách là cuốn sách bắt buộc phải lấy ra trước tiên nếu ta không muốn làm đổ cả chồng sách.

1.2 2. Nguyên lý hoạt động LIFO (Last-In, First-Out)

Nguyên lý LIFO quy định hai hành động cốt lõi:

- **Push (Đẩy vào):** Thêm một phần tử vào vị trí trên cùng của Stack (Top).
- **Pop (Lấy ra):** Loại bỏ phần tử đang nằm ở Top ra khỏi Stack.

1.3 3. Biểu diễn Stack bằng mảng (Array Representation)

Khi biểu diễn Stack bằng mảng một chiều tĩnh hoặc động:

- Ta cần một mảng chứa dữ liệu kích thước cực đại cố định là *max*.
- Một biến chỉ số *top* trỏ tới phần tử hiện tại nằm ở đỉnh Stack. Khi Stack rỗng, *top* được khởi tạo bằng -1 (theo quy chuẩn C chuẩn chỉ số bắt đầu từ 0).

1.4 4. Biểu diễn Stack bằng danh sách liên kết đơn (Singly Linked List Representation)

Khi biểu diễn Stack bằng DSLK đơn:

- Mỗi phần tử được cấp phát động trong vùng nhớ Heap dưới dạng một *Node*.
- Con trỏ *head* đóng vai trò là điểm Top của Stack.
- Thao tác Push tương đương với việc thêm một Node vào đầu danh sách (**Add at Head**), thao tác Pop tương đương lấy Node ở đầu ra (**Get at Head & Delete Head**).

1.5 5. Thao tác Khởi tạo Stack (Stack Initialization)

Dưới đây là phân tích chi tiết cho thao tác khởi tạo cho cả mảng và danh sách liên kết.

1.5.1 1. FLOWCHART KHỞI TẠO

[MẢNG STATIC / DYNAMIC]

+-----+

| Start |

+-----+

|

[reset array memory]

[set top = -1]

|

+-----+

| End |

+-----+

[DANH SÁCH LIÊN KẾT ĐƠN]

+-----+

| Start |

+-----+

|

[set head pointer]

[set head = NULL]

|

+-----+

| End |

+-----+

1.5.2 2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC

Trước khi khởi tạo, biến *top* hoặc con trỏ *head* chứa giá trị rác ngẫu nhiên từ phân vùng bộ nhớ của hệ thống:

Array: *top* = 0x7FFA94B (Rác) | Memory Array: [?, ?, ?, ?, ?]
 Singly Linked List: *head* = 0x0000FFAC (Rác)

1.5.3 3. SƠ ĐỒ ASCII SAU THAO TÁC

Array: *top* = -1 | Memory Array: [0, 0, 0, 0, 0] (Đã được dọn sạch)
 Singly Linked List: *head* = NULL (Trở về vùng an toàn rỗng)

1.5.4 4. PHÂN TÍCH THAY ĐỔI BIẾN

- **Với Mảng (Array):** Biến điều khiển *top* từ giá trị không xác định sẽ được gán giá trị tuyệt đối là -1 . Về mặt CPU, một thanh ghi chứa giá trị ghi đè -1 (thường biểu diễn dạng bù hai 0xFFFFFFFF) vào ô nhớ stack tương ứng của biến *top*.
- **Với DSLK (Linked List):** Con trỏ *head* kiểu cấu trúc Node* được gán bằng hằng số hệ thống NULL (giá trị số học là 0), đảm bảo hệ điều hành biết danh sách chưa liên kết tới bất kỳ ô nhớ Heap nào.

1.5.5 5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)

[VÙNG NHỚ STACK FRAME]

Địa chỉ ô nhớ: 0x0012FF40 | Tên biến: *top* | Giá trị: -1 (4 bytes)

Địa chỉ ô nhớ: 0x0012FF44 | Tên biến: *head* | Giá trị: 0x00000000 (NULL, 8 bytes)

1.5.6 6. CODE C CHUẨN KHỞI TẠO

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100 // Kích thước vực đại của mảng 1tnh
5
6 // Định nghĩa cấu trúc Stack dạng mảng
7 typedef struct {
8     int arr[MAX];
9     int top;
10 } ArrayStack;
11
12 // Khởi tạo Stack dạng mảng
13 void initArrayStack(ArrayStack *s) {
14     s->top = -1; // ếThit ập top ằng -1 để báo ệhiu ớng
15 }
16
17 // Định nghĩa cấu trúc Node cho DSLK Đơn
18 typedef struct Node {
19     int data;
20     struct Node *next;
21 } Node;
22
23 // Định nghĩa cấu trúc Stack dạng DSLK
24 typedef struct {
25     Node *head; // Con ỏtr ảqun lý đnh Stack

```

```

26 } ListStack;
27
28 // Ở Khi tạo Stack dùng DSLK
29 void initListStack(ListStack *s) {
30     s->head = NULL; // Thiết lập head về NULL ban đầu
31 }

```

1.5.7 7. CHECKLIST 30 GIÂY

- ☐ Mảng: Gán $top = -1$ chưa?
- ☐ DSLK: Gán $head = NULL$ chưa?
- ☐ Đã định nghĩa biến MAX an toàn chưa?

1.5.8 8. SAI LẦM THƯỜNG GẶP

⚠ CẢNH BÁO: Lỗi Khởi tạo

Lỗi: Không khởi tạo giá trị ban đầu cho top mà trực tiếp dùng các hàm $push$. Hệ thống sẽ đọc giá trị rác của top làm chỉ số mảng, gây ra lỗi **Segment Fault / Out of bounds access** ngay lập tức.

1.5.9 9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O

Cài đặt	Độ phức tạp thời gian	Độ phức tạp không gian
Mảng (Array)	$O(1)$	$O(MAX)$ (cấp phát tĩnh cố định)
DSLK (Linked List)	$O(1)$	$O(1)$ (chưa cấp phát động ô nhớ dữ liệu nào)

1.5.10 10. MẸO GHI NHỚ

💡 GHI NHỚ: Mẹo Nhớ

Nhớ quy luật: "Array thì trừ một ($top = -1$), List thì rỗng không ($head = NULL$)".

1.5.11 11. FLASHCARD CHUYÊN SÂU

Hỏi: Tại sao khi dùng mảng làm Stack, khởi tạo $top = -1$ thay vì 0?

Đáp: Vì chỉ số mảng trong C bắt đầu từ 0. Nếu gán $top = 0$ lúc khởi tạo, hệ thống sẽ hiểu đang có một phần tử hợp lệ ở chỉ số 0, dẫn tới sai lệch dữ liệu.

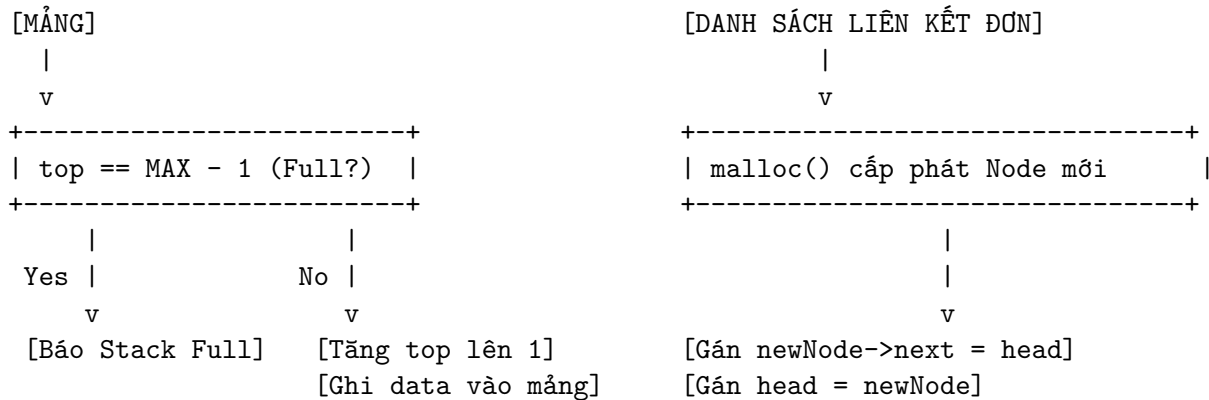
1.5.12 12. TÓM TẮT THAO TÁC KHỞI TẠO

- **Cơ chế cốt lõi:** Thiết lập lại trạng thái rỗng hoàn toàn để bắt đầu nạp phần tử.
- **Checklist nhanh:** Array $\rightarrow top = -1$; DSLK $\rightarrow head = NULL$.
- **Hiệu năng:** Luôn đạt $O(1)$ tức thì.

1.6 6. Thao tác Push (Thêm phần tử vào đỉnh Stack)

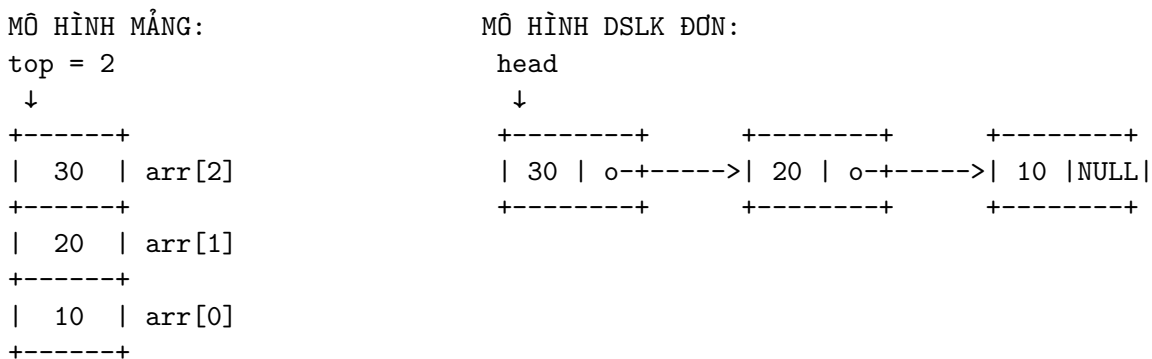
Thao tác cốt lõi đưa dữ liệu mới vào Stack. Ta phân tích cả hai mô hình biểu diễn.

1.6.1 1. FLOWCHART PUSH

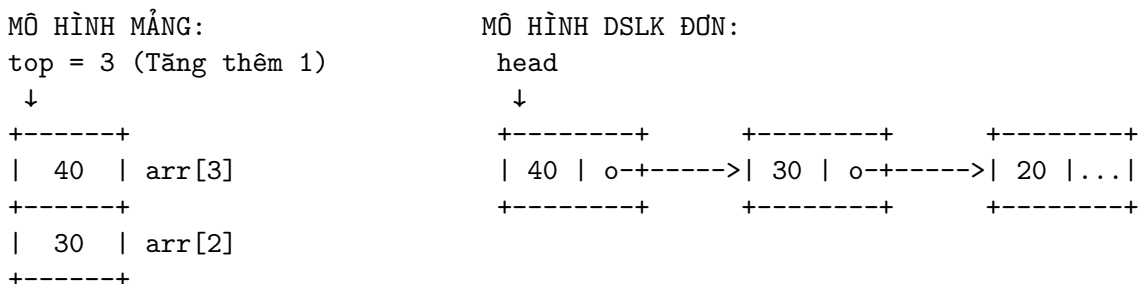


1.6.2 2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC

Thêm giá trị 40 vào Stack hiện có [10, 20, 30]:



1.6.3 3. SƠ ĐỒ ASCII SAU THAO TÁC



1.6.4 4. PHÂN TÍCH THAY ĐỔI BIẾN

- **Với Mảng:** 1. Kiểm tra trạng thái đầy mảng ($top == MAX - 1$). Nếu chưa đầy, thực hiện bước kế tiếp. 2. Tăng giá trị của top thêm 1 đơn vị (top chuyển từ 2 $\rightarrow$ 3). 3. Gán giá trị mới vào mảng tại vị trí chỉ số mới: $arr[top] = val$ ($arr[3] = 40$).
- **Với DSLK Đơn:** 1. Cấp phát động một ô nhớ có kích thước bằng cấu trúc Node đặt tên là $newNode$. 2. Gán trường dữ liệu: $newNode \rightarrow data = val$. 3. Thực hiện liên kết ngược: Đưa con trỏ liên kết của node mới trỏ thẳng vào Node đỉnh hiện tại: $newNode \rightarrow next = head$. 4. Cập nhật lại đỉnh Stack mới: $head = newNode$.

1.6.5 5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)

Khi cấp phát động cho DSLK, bộ nhớ Stack lưu giữ con trỏ head cục bộ trỏ sang vùng nhớ Heap được cấp phát thực tế:

[STACK FRAME]	[HEAP SEGMENT]
head = 0x00A1FF00	-----> Địa chỉ 0x00A1FF00 (Node 40):
	- data = 40
	- next = 0x00A1FE80 (Trỏ đến Node 30 cũ)

1.6.6 6. CODE C CHUẨN PUSH

```

1 // Thêm phần tử vào Stack mảng
2 int pushArray(ArrayStack *s, int val) {
3     // Theo tài liệu "3. slide Stack Queue.pdf" trang 4:
4     // Kiểm tra tràn Stack (Stack full!)
5     if (s->top == MAX - 1) {
6         printf("Stack full!\n");
7         return 0; // Trả về mã lỗi 0
8     }
9     s->top++;           // Tăng đỉnh stack một
10    s->arr[s->top] = val; // Đưa dữ liệu vào ô nhớ
11    return 1;           // Thành công
12 }
13
14 // Thêm phần tử vào Stack DSLK Đơn
15 int pushList(ListStack *s, int val) {
16     // Cấp phát một Node mới nằm trên Heap
17     Node *newNode = (Node*)malloc(sizeof(Node));
18     if (newNode == NULL) {
19         printf("Out of memory!\n");
20         return 0; // Trả về lỗi nếu hệ thống hết RAM
21     }
22     newNode->data = val; // Thiết lập dữ liệu node
23     newNode->next = s->head; // Liên kết node mới với đỉnh cũ
24     s->head = newNode; // Di chuyển head lên ở trỏ vào node mới
25     return 1;
26 }

```

1.6.7 7. CHECKLIST 30 GIÂY

- ☐ Với Mảng: Đã kiểm tra tràn mảng ($top == MAX - 1$)?
- ☐ Với Mảng: Đã tăng top trước khi gán dữ liệu?
- ☐ Với DSLK: Đã kiểm tra cấp phát động có bị NULL không?
- ☐ Với DSLK: Đã gán $newNode->next = head$ trước khi cập nhật head?

1.6.8 8. SAI LẦM THƯỜNG GẶP

❗ CẢNH BÁO: Lỗi Logic Cực Kỳ Nghiêm Trọng Trong Push DSLK

Lỗi: Gán ngược `head = newNode` trước khi liên kết `newNode->next = head`.

Hậu quả: Mất hoàn toàn địa chỉ của node đỉnh cũ dẫn đến **Rò rỉ bộ nhớ (Memory Leak)** diện rộng và biến Stack thành một danh sách đứt đoạn không thể truy cập các phần tử trước đó.

1.6.9 9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O

Cài đặt	Độ phức tạp thời gian	Độ phức tạp không gian
Mảng (Array)	$O(1)$	$O(1)$
DSLK (Linked List)	$O(1)$	$O(1)$ thêm mỗi lần đẩy dữ liệu

1.6.10 10. MẸO GHI NHỚ

❗ GHI NHỚ: Slogan ghi nhớ Push

"Mảng: Check tràn → Cộng đỉnh → Điền số.
List: Cấp phát → Đầu dây → Chuyển đầu."

1.6.11 11. FLASHCARD CHUYÊN SÂU

Hỏi: Ưu điểm lớn nhất của Push trên Linked List so với Array là gì?

Đáp: Linked List không bao giờ bị giới hạn bởi tham số kích thước cực đại MAX (Stack Full) trừ khi toàn bộ hệ thống máy chủ bị cạn kiệt tài nguyên RAM.

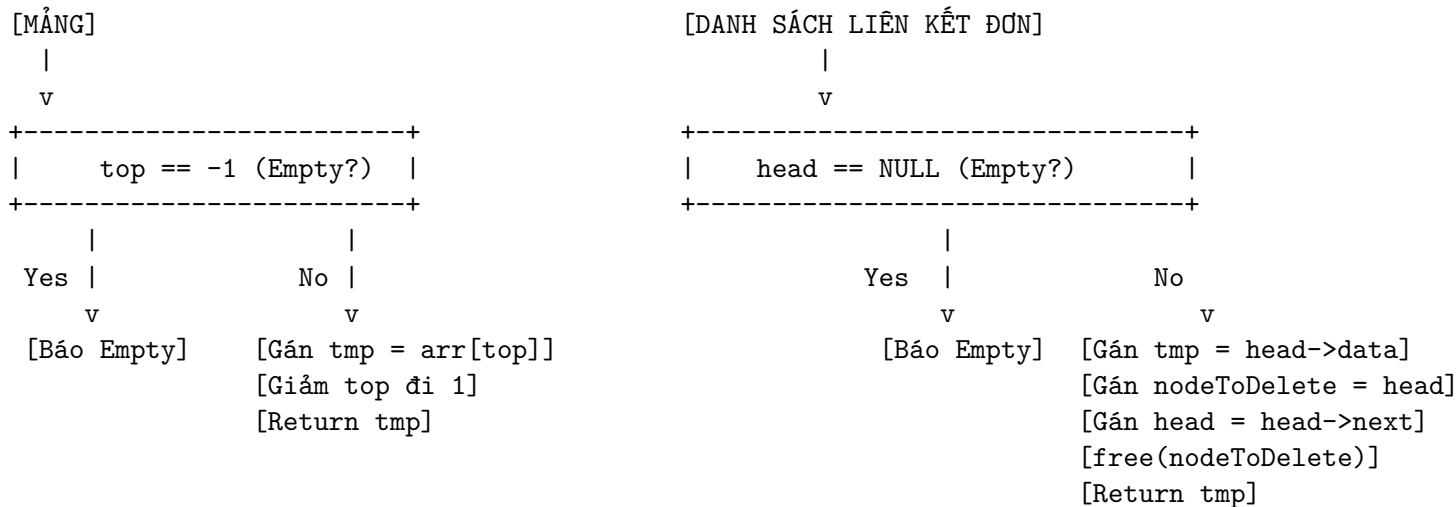
1.6.12 12. TÓM TẮT THAO TÁC PUSH

- **Cơ chế:** Đưa dữ liệu vào vị trí cao nhất (*top/head*).
- **Checklist:** Đảm bảo kiểm tra tính an toàn bộ nhớ của cấu trúc trước khi tác động con trỏ/chỉ số.

1.7 7. Thao tác Pop (Lấy và xóa phần tử khỏi đỉnh Stack)

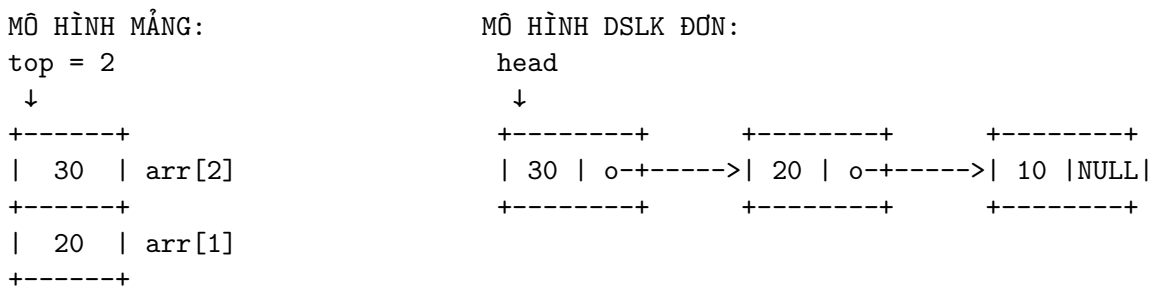
Thao tác rút trích dữ liệu ra khỏi Stack và dọn dẹp tài nguyên tương ứng.

1.7.1 1. FLOWCHART POP

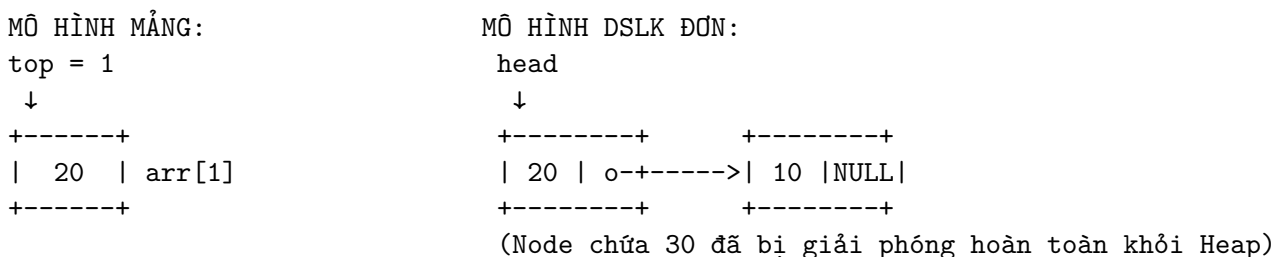


1.7.2 2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC

Ta tiến hành lấy phần tử ra khỏi Stack đang chứa [10, 20, 30]:



1.7.3 3. SƠ ĐỒ ASCII SAU THAO TÁC



1.7.4 4. PHÂN TÍCH THAY ĐỔI BIẾN

- **Với Mảng:** 1. Kiểm tra rỗng mảng ($top == -1$). Nếu rỗng báo lỗi và kết thúc. 2. Lưu trữ tạm thời giá trị đỉnh: $temp = arr[top]$. 3. Giảm biến đỉnh: $top = top - 1$. 4. Trả về giá trị của $temp$.
- **Với DSLK Đơn:** 1. Kiểm tra rỗng danh sách ($head == NULL$). 2. Tạo một con trỏ tạm để giữ địa chỉ Node đầu: $delNode = head$. 3. Trích xuất dữ liệu: $val = head->data$. 4. Di chuyển con trỏ đỉnh sang Node kế tiếp ở phía dưới: $head = head->next$. 5. Thu hồi ô nhớ Heap cũ: $free(delNode)$. 6. Trả về giá trị dữ liệu đã lấy.

1.7.5 5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)

Khi thực hiện `free()`, vùng nhớ nằm trên Heap được dọn dẹp sạch sẽ để tránh các tiến trình khác của hệ thống bị thiếu hụt bộ nhớ:

Trước `free()`: Heap chiếm dụng 24 bytes ở địa chỉ `0x00A1FF00`.

Sau `free()`: Vùng nhớ `0x00A1FF00` được trả về cho Hệ Điều Hành quản lý.

1.7.6 6. CODE C CHUẨN POP

```

1 // Pop array từ ô khi Stack mảng
2 int popArray(ArrayStack *s) {
3     // Theo tài liệu "3. slide Stack Queue.pdf" trang 5:
4     // Kiểm tra rỗng (Stack is empty!)
5     if (s->top == -1) {
6         printf("Stack is empty!\n");
7         return -1; // Trả về mã lỗi xác định
8     }
9     int tmp = s->arr[s->top]; // Lấy giá trị đỉnh vào biến tmp
10    s->top--; // Giảm 1 ô top của mảng
11    return tmp; // Trả về array từ lấy ra
12 }
13
14 // Pop array từ ô khi Stack DSLK Đơn
15 int popList(ListStack *s) {
16     // Theo tài liệu "3. slide Stack Queue.pdf" trang 5 (thao tác getAtHead)
17     if (s->head == NULL) {
18         printf("List is empty!\n");
19         return -1; // Trả về mã lỗi -1
20     }
21     Node *tmpNode = s->head; // Lấy địa chỉ node đầu tiên
22     int tmpData = tmpNode->data; // Lấy dữ liệu
23     s->head = s->head->next; // Chuyển head về node phía sau
24     free(tmpNode); // Thu hồi vùng nhớ Heap của node bị lấy ra
25     return tmpData; // Trả về dữ liệu
26 }

```

1.7.7 7. CHECKLIST 30 GIÂY

- ☐ Đã kiểm tra điều kiện rỗng (`top == -1` hoặc `head == NULL`)?
- ☐ Đã sao lưu dữ liệu vào biến tạm trước khi di chuyển biến đỉnh?
- ☐ Với DSLK: Đã lưu địa chỉ Node cần giải phóng vào biến con trỏ tạm chưa?
- ☐ Với DSLK: Đã thực hiện `free()` thành công chưa?

1.7.8 8. SAI LẦM THƯỜNG GẶP

⚠ CẢNH BÁO: Lỗi Rò rỉ Bộ Nhớ Lớn Khi Dùng DSLK

Sai lầm: Thực hiện dịch chuyển `head = head->next` mà quên mất bước `free(tmpNode)`. Chương trình vẫn chạy đúng kết quả nhưng lượng RAM tiêu thụ sẽ tăng liên tục theo thời gian cho tới khi hệ thống bị treo.

1.7.9 9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O

Cài đặt	Độ phức tạp thời gian	Độ phức tạp không gian
Mảng (Array)	$O(1)$	$O(1)$
DSLK (Linked List)	$O(1)$	$O(1)$

1.7.10 10. MẸO GHI NHỚ

▣ GHI NHỚ: Mnemonic Pop

"Pop là rút: Mảng giảm index ($top - -$). List cắt con trỏ ($head = head -> next$), giải phóng vùng nhớ ($free$)."

1.7.11 11. FLASHCARD CHUYÊN SÂU

Hỏi: Tại sao thao tác Pop trên mảng không cần xóa dữ liệu cũ trong mảng mà chỉ giảm chỉ số top ?

Đáp: Vì khi giảm top , vùng nhớ của phần tử cũ trở thành vùng nhớ không hợp lý đối với hoạt động của Stack. Nếu có lệnh Push kế tiếp, giá trị mới sẽ ghi đè trực tiếp lên vị trí dữ liệu cũ đó.

1.7.12 12. TÓM TẮT THAO TÁC POP

- **Cơ chế:** Lấy ra phần tử đầu đỉnh Stack, giảm kích thước tổng thể đi 1.
- **Chìa khóa thành công:** Kiểm tra trống trước khi thao tác và luôn dọn bộ nhớ đối với cấp phát động.

1.8 8. Thao tác bổ trợ Stack: Peek, isEmpty, isFull

Chúng ta nhanh chóng xem xét mã nguồn tối ưu của các phương thức này.

```

1 // Ầy giá trị đỉnh nhưng không xóa (Peek / Top)
2 int peekArray(ArrayStack *s) {
3     if (s->top == -1) {
4         printf("Stack empty!\n");
5         return -1;
6     }
7     return s->arr[s->top];
8 }
9
10 int peekList(ListStack *s) {
11     if (s->head == NULL) {
12         printf("Stack empty!\n");
13         return -1;
14     }
15     return s->head->data;
16 }
17
18 // ểKim tra Stack ỗng (isEmpty)
19 int isEmptyArray(ArrayStack *s) {
20     return (s->top == -1);
21 }
22
23 int isEmptyList(ListStack *s) {
24     return (s->head == NULL);
25 }
26
27 // ểKim tra Stack đầy (isFull)
28 int isFullArray(ArrayStack *s) {
29     return (s->top == MAX - 1);
30 }
31
32 int isFullList(ListStack *s) {
33     return 0; // Danh sách liên ểkt luôn không ịb đầy trên lý ểthuyết
34 }

```

1.9 9. Ứng dụng thực tế của Stack

Stack là trái tim của rất nhiều thuật toán kinh điển trong khoa học máy tính:

- **Kiểm tra tính hợp lệ của biểu thức toán học:** Kiểm tra xem các dấu đóng mở ngoặc như { [()] } có khớp cặp hay không (được nêu bật trong trang 6 tài liệu "[3. slide Stack Queue.pdf](#)").
- **Khử đệ quy (Recursion elimination):** Bản chất hệ thống sử dụng cấu trúc *System Stack Frame* để lưu trữ thông tin lời gọi hàm. Ta có thể mô phỏng lại quá trình này bằng Stack tự tạo để chuyển đổi một bài toán đệ quy sang vòng lặp.
- **Tính toán biểu thức:** Chuyển đổi biểu thức dạng Trung tố (Infix) sang Hậu tố (Postfix) và thực thi tính toán biểu thức Hậu tố một cách tuần tự.

1.10 10. Các lỗi lập trình viên thường gặp với Stack

⚠ CẢNH BÁO: Bẫy Lập Trình Stack

1. **Stack Overflow (Tràn bộ nhớ hệ thống):** Do đệ quy vô hạn tạo quá nhiều Stack Frame mà không có điểm dừng.
2. **Stack Underflow (Trích xuất rỗng):** Gọi lệnh Pop liên tục khi Stack đã trống không. Luôn luôn phải sử dụng hàm kiểm tra `isEmpty()` trước khi thực thi lệnh lấy dữ liệu ra ngoài.

2 PHẦN II: QUEUE (HÀNG ĐỢI)

2.1 1. Khái niệm & Đặc tả toán học

Queue (Hàng đợi) là cấu trúc danh sách tuyến tính hoạt động nghiêm ngặt theo quy tắc thứ tự thời gian.

Trang 7 của tài liệu gốc "**3. slide Stack Queue.pdf**" mô phỏng trực quan dòng người xếp hàng tại quầy thanh toán siêu thị: người nào xếp hàng trước nhất sẽ được xử lý thanh toán trước và ra về trước.

2.2 2. Nguyên lý FIFO (First-In, First-Out)

Nguyên lý FIFO ("Vào trước ra trước") định nghĩa rõ hai đầu riêng biệt:

- **Enqueue (Xếp hàng):** Đút một phần tử mới vào cuối hàng đợi (**Rear/End**).
- **Dequeue (Rời hàng):** Lấy phần tử ở đầu hàng đợi (**Front/Left**) ra xử lý.

2.3 3. Biểu diễn Queue bằng Mảng (Array Representation)

Biểu diễn một Queue bằng mảng đòi hỏi nhiều tham số quản lý hơn Stack. Theo trang 9 của tài liệu "**3. slide Stack Queue.pdf**":

- Mảng tĩnh `arr` có tối đa `MAX` phần tử.
- Chỉ số `left` (hay `front`): Quản lý chỉ số bắt đầu của Queue.
- Chỉ số `right` (hay `rear`): Quản lý chỉ số kết thúc của Queue.
- Biến `count` (hay `size`): Lưu trữ số lượng phần tử hiện tại nhằm kiểm soát chính xác trạng thái trống đầy.

2.4 4. Biểu diễn Queue bằng danh sách liên kết đơn (Singly Linked List Representation)

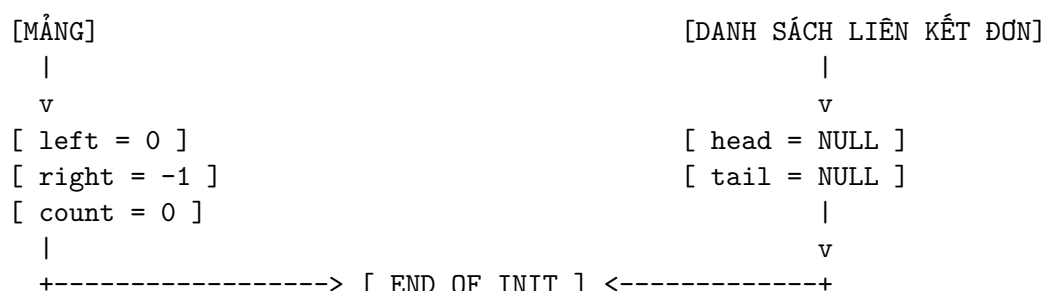
Với DSLK Đơn, để đạt hiệu năng $O(1)$ cho cả hai đầu, ta lưu trữ song song hai con trỏ quản lý:

- Con trỏ `head` (hoặc `front`) trỏ vào nút đầu tiên (phục vụ Dequeue).
- Con trỏ `tail` (hoặc `rear`) trỏ thẳng vào nút cuối cùng của danh sách (phục vụ Enqueue nhanh chóng không cần duyệt từ đầu đến cuối).

2.5 5. Thao tác Khởi tạo Queue

Chúng ta tiến hành khởi động cấu trúc dữ liệu hàng đợi.

2.5.1 1. FLOWCHART KHỞI TẠO QUEUE



2.5.2 2. SƠ ĐỒ ASCII TRƯỚC VÀ SAU THAO TÁC

Khởi tạo dọn dẹp tất cả các giá trị rác trong bộ nhớ để thiết lập trạng thái ban đầu ổn định:

Trước khởi tạo: left, right, count chứa các dữ liệu số học ngẫu nhiên bần.

Sau khởi tạo:

Array Queue: left = 0, right = -1, count = 0

Linked List Queue: head = NULL, tail = NULL

2.5.3 3. CODE C CHUẨN KHỞI TẠO QUEUE

```

1 #define QMAX 100 // Kích thước ô ghi nhận của Queue mảng
2
3 // Định nghĩa Queue cấu trúc mảng
4 typedef struct {
5     int arr[QMAX];
6     int left; // Chỉ số phần tử bắt đầu đầu dòng
7     int right; // Chỉ số phần tử kết thúc cuối dòng
8     int count; // Số lượng phần tử hiện hữu
9 } ArrayQueue;
10
11 // Khi tạo Queue mảng theo slide 9 tài liệu "3. slide Stack Queue.pdf"
12 void initArrayQueue(ArrayQueue *q) {
13     q->left = 0;
14     q->right = -1;
15     q->count = 0;
16 }
17
18 // Định nghĩa Queue cấu trúc DSLK Đơn
19 typedef struct {
20     Node *head; // Con trỏ đầu Front
21     Node *tail; // Con trỏ cuối Rear
22 } ListQueue;
23
24 // Khi tạo Queue DSLK Đơn
25 void initListQueue(ListQueue *q) {
26     q->head = NULL;
27     q->tail = NULL;
28 }

```

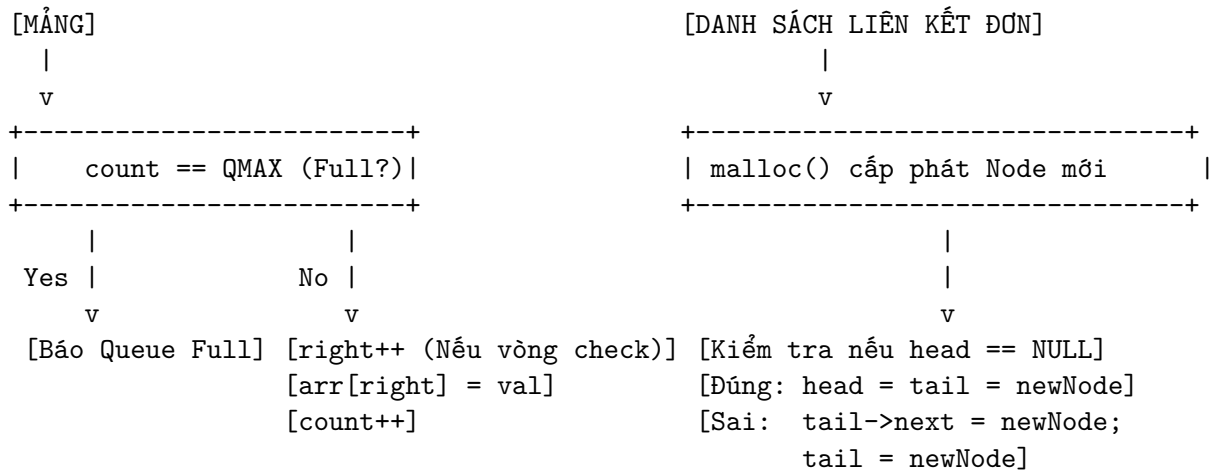
2.5.4 4. CHECKLIST 30 GIÂY

- ☐ Mảng: left = 0, right = -1, count = 0 chưa?
- ☐ DSLK: Cả hai con trỏ head và tail đã về NULL chưa?

2.6 6. Thao tác EnQueue (Thêm phần tử vào cuối hàng đợi)

Quá trình đưa dữ liệu mới xếp vào hàng.

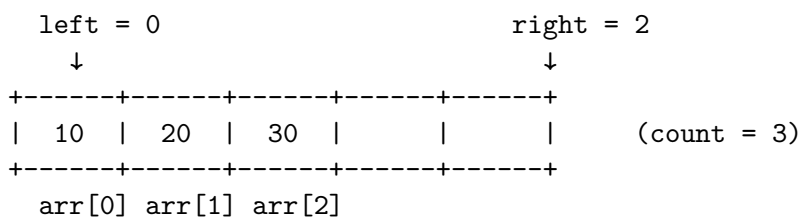
2.6.1 1. FLOWCHART ENQUEUE



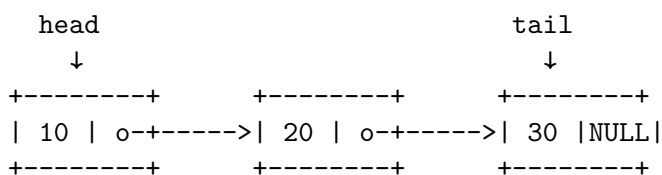
2.6.2 2. SƠ ĐỒ ASCII TRƯỚC THAO TÁC ENQUEUE

Thêm phần tử 40 vào cuối hàng đợi hiện tại đang chứa [10, 20, 30]:

MÔ HÌNH MẢNG:

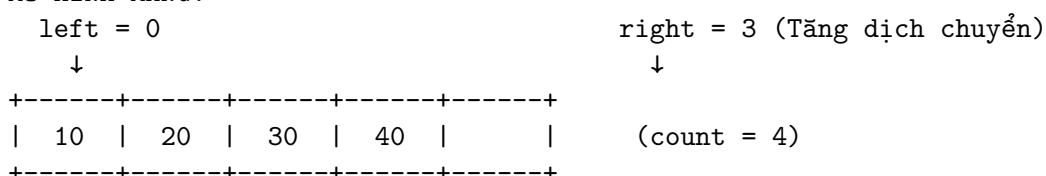


MÔ HÌNH DSLK ĐƠN:

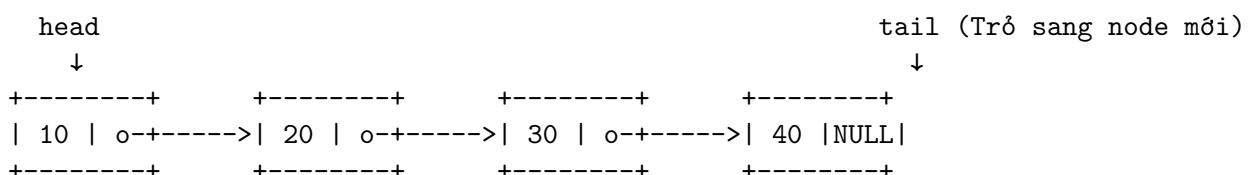


2.6.3 3. SƠ ĐỒ ASCII SAU THAO TÁC ENQUEUE

MÔ HÌNH MẢNG:



MÔ HÌNH DSLK ĐƠN:



2.6.4 4. PHÂN TÍCH THAY ĐỔI BIẾN

- **Với Mảng:** 1. Kiểm tra tràn mảng (`count == QMAX`). Nếu không tràn thực hiện bước tiếp theo. 2. Tăng giá trị chỉ số cuối `right` thêm 1 (chuyển dịch từ 2 $\rightarrow$ 3). 3. Điền giá trị mới vào mảng: `arr[3] = 40`. 4. Tăng tổng biến đếm số lượng: `count` chuyển từ 3 $\rightarrow$ 4.
- **Với DSLK Đơn:** 1. Tạo và nạp dữ liệu cho `newNode`. 2. Nếu danh sách đang rỗng rành rành (`head == NULL`), cả hai con trỏ điểm đầu và cuối cùng đồng loạt trỏ vào node mới này. 3. Nếu đã có phần tử trước đó: Liên kết node cuối cũ tới node mới (`tail->next = newNode`), sau đó di chuyển định danh cuối (`tail = newNode`).

2.6.5 5. MEMORY VIEW (GÓC NHÌN BỘ NHỚ)

Vùng nhớ Heap được liên kết nối đuôi thông qua con trỏ liên kết của nút Tail cũ: `[tail->next]` chứa địa chỉ vật lý của Node mới cấp phát trên Heap.

2.6.6 6. CODE C CHUẨN ENQUEUE

```

1 // Enqueue đối với mảng tĩnh để tính toán thông thường
2 int enqueueArray(ArrayQueue *q, int val) {
3     // Theo thuật toán trang 10 của tài liệu "3. slide Stack Queue.pdf"
4     if (q->count == QMAX) {
5         printf("Queue full!\n");
6         return 0; // Tràn hàng đợi
7     }
8     // Thiết lập tăng để tính
9     if (q->right == QMAX - 1) {
10         q->right = 0; // ÝK thuật vòng xoay mảng (Circular)
11     } else {
12         q->right++;
13     }
14     q->arr[q->right] = val; // Đưa dữ liệu vào ô nhớ đích
15     q->count++;           // Tăng số lượng đếm phần tử
16     return 1;
17 }
18
19 // Enqueue đối với cấu trúc danh sách liên kết sử dụng tail tối ưu O(1)
20 int enqueueList(ListQueue *q, int val) {
21     Node *newNode = (Node*)malloc(sizeof(Node));
22     if (newNode == NULL) return 0;
23     newNode->data = val;
24     newNode->next = NULL;
25
26     if (q->head == NULL) {
27         // Hàng đợi trống ban đầu
28         q->head = newNode;
29         q->tail = newNode;
30     } else {
31         q->tail->next = newNode; // Đầu liên kết nối tiếp vào đuôi cũ
32         q->tail = newNode;      // để chuyển dịch con trỏ tail quản lý
33     }
34     return 1;
35 }

```

2.6.7 7. CHECKLIST 30 GIÂY

- ☐ Mảng: Đã kiểm tra trạng thái đầy hàng (`count == QMAX`) chưa?

- ☐ Mảng: Đã tăng chỉ số `right` và tăng biến đếm `count` chưa?
- ☐ DSLK: Đã xử lý phân tách trường hợp đặc biệt khi danh sách rỗng ban đầu chưa?
- ☐ DSLK: Đã gán `newNode->next = NULL` để tránh rác liên kết?

2.6.8 8. SAI LẦM THƯỜNG GẶP

⚠ CẢNH BÁO: Lỗi Tràn Ảo Trên Mảng Tuyến Tính

Hiện tượng: Enqueue và Dequeue liên tục làm cho biến chỉ số `right` chạm mốc `MAX-1` trong khi phía trước của mảng đã trống rỗng hoàn toàn do Dequeue giải phóng. Hệ thống báo lỗi tràn mảng mặc dù dung lượng mảng thực tế vẫn còn trống rất nhiều. **Giải pháp khắc phục bắt buộc:** Áp dụng cấu trúc **Circular Queue (Hàng đợi vòng tròn)** sử dụng phép toán chia lấy dư Modulo.

2.6.9 9. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O

Cài đặt	Độ phức tạp thời gian	Độ phức tạp không gian
Mảng (Array)	$O(1)$	$O(1)$
DSLK (Linked List)	$O(1)$ (nếu lưu trữ con trỏ <code>tail</code>)	$O(1)$

2.6.10 10. MẸO GHI NHỚ

💡 GHI NHỚ: Thần chú Enqueue

"Enqueue là nạp cuối đuôi: Mảng tăng `right`, tăng `count`. List móc thêm node mới vào vị trí `tail->next` rồi nhảy `tail` sang."

2.6.11 11. FLASHCARD CHUYÊN SÂU

Hỏi: Tại sao trong DSLK của Queue nếu không dùng con trỏ `tail` thì độ phức tạp thao tác Enqueue sẽ bị giảm sút?

Đáp: Vì nếu không có `tail`, ta bắt buộc phải dùng vòng lặp duyệt từ `head` đến phần tử cuối cùng để tìm địa chỉ đuôi, đẩy độ phức tạp thời gian từ cực ưu tú $O(1)$ thành tồi tệ $O(N)$.

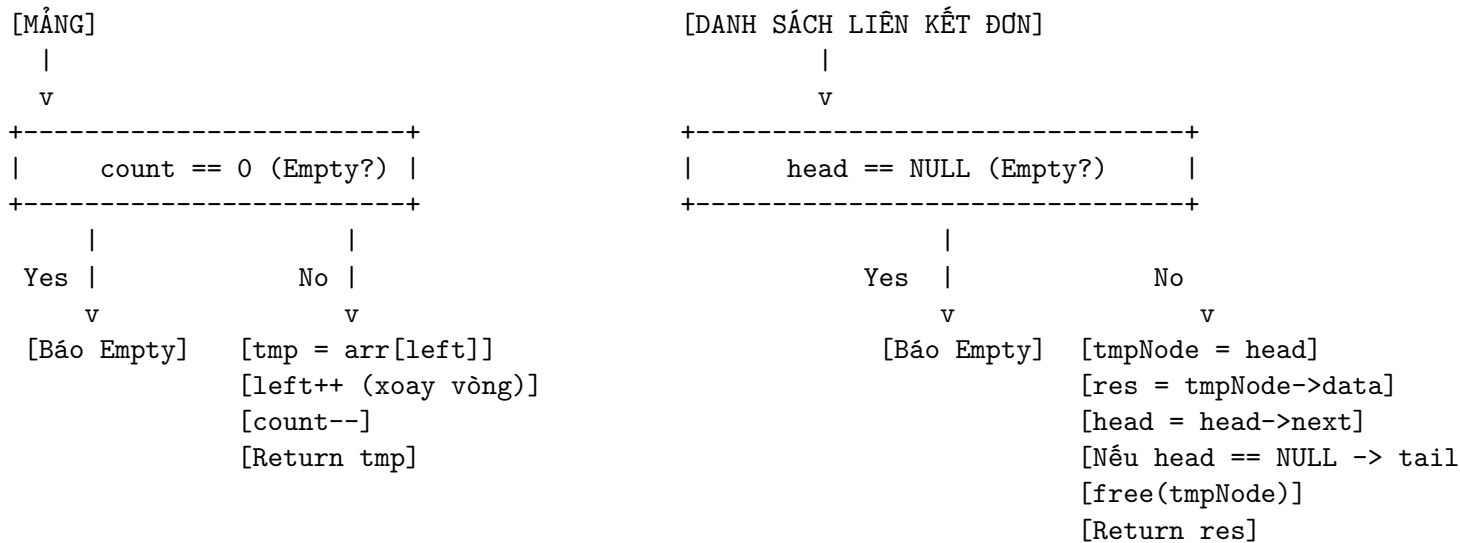
2.6.12 12. TÓM TẮT THAO TÁC ENQUEUE

- **Cơ chế:** Chèn thêm phần tử vào điểm đuôi của hàng đợi.
- **Nguyên tắc:** Luôn kiểm tra tính toàn vẹn của chỉ số biên mảng và con trỏ `tail` rỗng.

2.7 7. Thao tác DeQueue (Lấy phần tử ra khỏi đầu hàng đợi)

Quá trình xử lý và đưa phần tử rời khỏi hàng theo đúng nguyên lý xếp hàng.

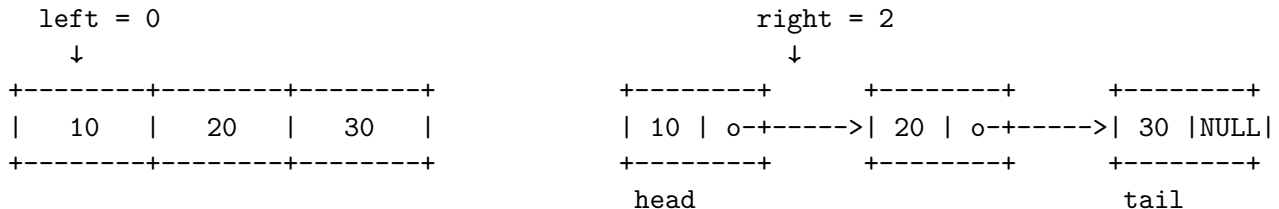
2.7.1 1. FLOWCHART DEQUEUE



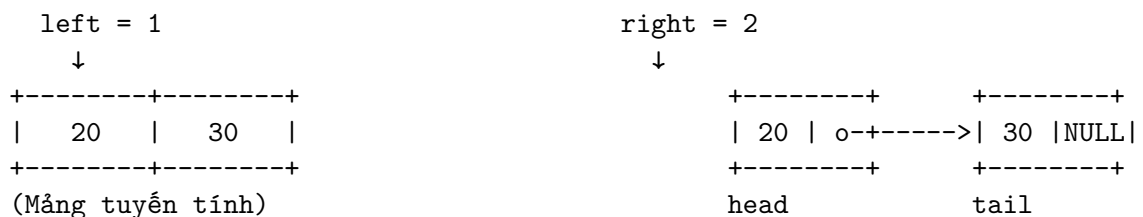
2.7.2 2. SƠ ĐỒ ASCII TRƯỚC VÀ SAU DEQUEUE

Tiến hành Dequeue lấy giá trị đầu ra khỏi Queue chứa [10, 20, 30]:

TRƯỚC DEQUEUE:



SAU DEQUEUE:



2.7.3 3. CODE C CHUẨN DEQUEUE

```

1 // Dequeue đối với mảng tĩnh
2 int dequeueArray(ArrayQueue *q) {
3     // Theo trang 11 tài liệu "3. slide Stack Queue.pdf"
4     if (q->count == 0) {
5         printf("Queue empty!\n");
6         return -1; // Hàng đợi trống
7     }
8     int tmp = q->arr[q->left]; // lấy giá trị đầu
9
10    // tăng vòng index ở left

```

```

11     if (q->left == QMAX - 1) {
12         q->left = 0;
13     } else {
14         q->left++;
15     }
16     q->count--; // Giảm số đếm lượng phần tử
17     return tmp;
18 }
19
20 // Dequeue đối với DSLK Đơn ược chuẩn giải phóng bộ nhớ asch es
21 int dequeueList(ListQueue *q) {
22     if (q->head == NULL) {
23         printf("Queue empty!\n");
24         return -1;
25     }
26     Node *tmpNode = q->head; // Lấy địa chỉ node đầu đỉnh Front
27     int resData = tmpNode->data; // Trích xuất thông tin
28
29     q->head = q->head->next; // Đẩy head sang node kế tiếp
30
31     // Nếu sau khi dịch chuyển mà hàng trống thì đồng thời cập nhật tail
32     if (q->head == NULL) {
33         q->tail = NULL;
34     }
35
36     free(tmpNode); // Giải phóng bộ nhớ trên Heap tránh Memory Leak
37     return resData;
38 }

```

2.7.4 4. CHECKLIST 30 GIÂY

- ☐ Đã kiểm tra trạng thái rỗng hàng (count == 0 hoặc head == NULL) chưa?
- ☐ Mảng: Đã tăng tính tiến chỉ số left và giảm lượng count chưa?
- ☐ DSLK: Đã xử lý kéo con trỏ tail về NULL khi nút cuối cùng bị rút ra khỏi hàng chưa?
- ☐ DSLK: Đã thực hiện giải phóng tài nguyên ô nhớ động của Node rách bằng lệnh free()?

2.7.5 5. BẢNG PHÂN TÍCH ĐỘ PHỨC TẠP BIG O DEQUEUE

Cài đặt	Độ phức tạp thời gian	Độ phức tạp không gian
Mảng (Array)	$O(1)$	$O(1)$
DSLK (Linked List)	$O(1)$	$O(1)$

2.8 8. Circular Queue (Hàng đợi vòng tròn)

Để giải quyết triệt để lỗi **"Tràn ảo"** của Queue mảng tuyến tính, ta cuộn tròn mảng lại. Vị trí chỉ số mảng kế tiếp của i không chỉ đơn thuần là $i + 1$ nữa mà sẽ được tính toán bởi phép toán vòng tròn Modulo: $(i + 1) \% QMAX$.

```

1 // ÝK thuật Enqueue vòng tròn chuẩn xác toán học máy tính
2 int enqueueCircular(ArrayQueue *q, int val) {
3     if (q->count == QMAX) {
4         printf("Queue is full!\n");
5         return 0;
6     }
7     // Dùng toán tử modulo để nhay vòng tròn ích ỗ ết thức
8     q->right = (q->right + 1) % QMAX;
9     q->arr[q->right] = val;
10    q->count++;
11    return 1;
12 }
13
14 // ÝK thuật Dequeue vòng tròn
15 int dequeueCircular(ArrayQueue *q) {
16     if (q->count == 0) {
17         printf("Queue is empty!\n");
18         return -1;
19     }
20     int tmp = q->arr[q->left];
21     // ộCun tròn ích ỗ ất đầu đầu dòng
22     q->left = (q->left + 1) % QMAX;
23     q->count--;
24     return tmp;
25 }

```

2.9 9. Priority Queue (Hàng đợi ưu tiên)

Định nghĩa mô tả trong trang 12 của tài liệu gốc **"3. slide Stack Queue.pdf"**: Hàng đợi ưu tiên là dạng hàng đợi nâng cao mà các phần tử đi kèm một thông số chỉ số **"Độ ưu tiên" (Priority)**. Phần tử có chỉ số ưu tiên cao hơn sẽ tự động được đưa lên xếp trước đầu hàng để rút ra trước tiên dù thời gian nạp vào sau.

```

1 // Định nghĩa Node kèm độ u tiên cho Priority Queue ạng DSLK Đơn
2 typedef struct PQNode {
3     int data;
4     int priority; // Độ u tiên: Giá trị ỗ ốnh ểbiu ịth ứmc độ u tiên càng ớn
5     struct PQNode *next;
6 } PQNode;
7
8 // Enqueue có ắp ếp theo độ u tiên ắtng ầu
9 void enqueuePriority(PQNode **head, int data, int p) {
10    PQNode *newNode = (PQNode*)malloc(sizeof(PQNode));
11    newNode->data = data;
12    newNode->priority = p;
13    newNode->next = NULL;
14
15    // ườTrng ợp đầu hàng ốtrng ặhoc node ỡm có u tiên cao ớn node đầu ệhìn ạt i
16    if (*head == NULL || (*head)->priority > p) {
17        newNode->next = *head;
18        *head = newNode;
19    } else {
20        // ệDuyt tìm ịv trí thích ợp để chèn ữg nguyên ứth ựt u tiên

```

```
21     PQNode *temp = *head;
22     while (temp->next != NULL && temp->next->priority <= p) {
23         temp = temp->next;
24     }
25     newNode->next = temp->next;
26     temp->next = newNode;
27 }
28 }
```

2.10 10. Ứng dụng của Queue trong thực tế

Hàng đợi đóng vai trò quyết định trong xử lý bất đồng bộ và đồng bộ luồng:

- **Hệ thống Đệm (Buffers):** Quản lý bộ đệm in ấn (Printer Spooler), truyền dẫn gói tin đường truyền mạng TCP/IP.
- **Lập lịch CPU (CPU Scheduling):** Giải thuật Round-Robin lập lịch công việc cho nhân chip xử lý đa nhiệm.
- **Thuật toán đồ thị:** Tìm kiếm theo chiều rộng (BFS - Breadth-First Search) bắt buộc sử dụng Queue để quản lý tập hợp đỉnh chờ duyệt.

3 PHẦN III: CHƯƠNG TỔNG HỢP & SO SÁNH TOÀN DIỆN

Để hệ thống hóa và củng cố toàn bộ tri thức ngăn xếp và hàng đợi, chúng ta cùng phân tích sâu các bài toán thực tiễn và đưa ra bảng tổng hợp tổng lực để làm tài liệu tra cứu cấp tốc.

3.1 1. So sánh chi tiết triết lý Stack và Queue

Tiêu chí	Stack (Ngăn xếp)	Queue (Hàng đợi)
Nguyên lý	LIFO (Last-In First-Out) - Vào sau ra trước	FIFO (First-In First-Out) - Vào trước ra trước
Đầu tác động	Chỉ thao tác tại duy nhất 1 đầu (Top)	Thêm ở đầu cuối (Rear), lấy ra ở đầu trước
Khái niệm đời thực	Chồng đĩa, chồng sách, lịch sử lướt Web	Dòng người xếp hàng mua vé, gửi email in
Tìm kiếm phần tử	$O(N)$	$O(N)$

3.2 2. Ưu và nhược điểm khi lựa chọn Array vs Linked List để cài đặt

- Cài đặt bằng Mảng (Array):
- *Ưu điểm:* Tốc độ truy cập cực kỳ nhanh do dữ liệu nằm liền kề vật lý trên RAM (Cache Locality cao), không tốn thêm bộ nhớ để lưu giữ con trỏ next.

- *Nhược điểm:* Bị giới hạn kích thước cố định, nguy cơ lãng phí bộ nhớ nếu khai báo quá to hoặc lỗi tràn bộ nhớ nếu dữ liệu thực tế vượt ngưỡng dự tính ban đầu.
- Cài đặt bằng Danh sách liên kết (Linked List):
- *Ưu điểm:* Co giãn kích thước vô hạn tự động linh hoạt, tận dụng tối đa các vùng nhớ trống rải rác trên Heap.

- *Nhược điểm:* Tốn thêm bộ nhớ RAM cho các biến con trỏ (8 bytes trên hệ thống 64-bit cho mỗi liên kết), tốn thêm chi phí thời gian thực hiện lệnh gọi cấp phát động hệ thống malloc / giải phóng free.

3.3 3. Bảng tổng hợp độ phức tạp thời gian (Time Complexity) và không gian (Space)

Cấu trúc	Thao tác	Thời gian (Mảng)	Thời gian (DSLK)	Bộ nhớ phụ trợ
Stack	Push	$O(1)$	$O(1)$	$O(1)$
Stack	Pop	$O(1)$	$O(1)$	$O(1)$
Queue	Enqueue	$O(1)$ (vòng tròn)	$O(1)$ (có đuôi)	$O(1)$
Queue	Dequeue	$O(1)$ (vòng tròn)	$O(1)$	$O(1)$

3.4 4. Khi nào nên áp dụng loại nào?

□ GHI NHỚ: Kim chỉ nam thiết kế kiến trúc hệ thống

- **Dùng Stack khi:** Bạn cần thực hiện quay lui (Backtracking), khôi phục trạng thái cũ (Undo/Redo), phân tích cú pháp dấu ngoặc toán học, tính toán cây biểu thức.
- **Dùng Queue khi:** Cần truyền tải tuần tự giữa hai luồng độc lập có chênh lệch tốc độ truyền nhận, hàng đợi xử lý lệnh sự kiện trong lập trình UI, điều phối tài nguyên CPU.
- **Dùng Circular Queue khi:** Muốn cài đặt Queue trên mảng tĩnh với dung lượng giới hạn để đạt tốc độ xử lý phần cứng cao nhất mà không sợ rủi ro tràn ảo.
- **Dùng Priority Queue khi:** Hệ thống lập lịch tác vụ cần ưu tiên tiến trình khẩn cấp của hệ điều hành, thuật toán tìm đường đi ngắn nhất Dijkstra.